MINI PROJECT
GENERATIVE AI by IBM

Name: Aravind Krishnan
CS/AI-B

PYTHON

Quick Study Guide • Beginner to Early Intermediate

Goal: Learn the core ideas needed to read, write, and debug small Python programs.

How to use this guide

- Read one section at a time and type the examples into a Python interpreter.
- Change values, predict the output, then run the code.
- Practise without looking and explain each line in your own words.

What is Python?

Python is a general-purpose programming language known for readable syntax. It is used in automation, web development, data analysis, AI, scripting, and education.

Your first program

```
print("Hello, world!")
name = input("What is your name? ")
print("Hello,", name)
```

print() displays output; input() reads text typed by the user.

Good habits

- Use consistent 4-space indentation inside blocks.
- Names are case-sensitive: score and Score differ.
- Comments begin with #; explain why code exists.

1. Variables, types, and operators

A variable is a name that refers to a value. Python determines the type from the value assigned.

```
name = "Maya"        # str: text
age = 18             # int: whole number
height = 1.65        # float: decimal number
is_student = True    # bool: True or False
nothing = None       # absence of a value
```

Common types

Type	Example	Use
int	42	Whole numbers
float	3.14	Decimal numbers
str	'hello'	Text
bool	True	Logical conditions
list	[1, 2, 3]	Ordered, changeable collection
tuple	(1, 2)	Ordered, fixed collection
dict	{'name': 'Maya'}	Key–value pairs
set	{1, 2, 3}	Unique values

Operators

```
7 + 2   # 9        7 - 2   # 5
7 * 2   # 14       7 / 2   # 3.5
7 // 2  # 3        7 % 2   # 1 (remainder)
2 ** 3  # 8

# Comparisons: ==  !=  <  <=  >  >=
# Logic: and  or  not
```

Remember: Use = to assign a value; use == to compare values.

2. Decisions and loops

Conditions: if / elif / else

```
temperature = 29
if temperature >= 35:
    print("Very hot")
elif temperature >= 25:
    print("Warm")
else:
    print("Cool")
```

The first true condition runs. A colon starts a block; indentation defines the block.

for loops

```
for number in range(1, 5):
    print(number)  # 1, 2, 3, 4
```

range(start, stop) includes the start but excludes the stop.

while loops

```
count = 3
while count > 0:
    print(count)
    count -= 1
print("Go!")
```

Ensure a while loop can stop; otherwise it may run forever.

Loop control

- break exits the nearest loop.
- continue skips to the next iteration.

Mini example: sum numbers 1 to 5

```
total = 0
for number in range(1, 6):
    total += number
print(total)  # 15
```

Trace loops by writing down variable values after each iteration. This is a useful debugging technique.

3. Collections and strings

Lists

```
fruits = ["apple", "mango", "pear"]
print(fruits[0])        # apple; indexing starts at 0
fruits.append("kiwi")
fruits[1] = "orange"
print(len(fruits))
```

- Useful methods: append(), remove(), sort().
- A slice such as fruits[0:2] returns items at indexes 0 and 1.

Dictionaries

```
student = {"name": "Ari", "score": 92}
print(student["name"])
student["score"] = 95
student["grade"] = "A"
```

A dictionary maps keys to values; use a key to look up, update, or add information.

Strings

```
message = "Python is fun"
print(message.lower())
print(message.upper())
print(message.replace("fun", "useful"))
print(message[0:6])  # Python
name = "Ari"
print(f"Hello, {name}!")
```

Strings are immutable: string methods return a new string rather than changing the original.

Choose the right collection

- List: ordered items that may change.
- Tuple: ordered items intended to stay fixed.
- Set: unique items; useful for removing duplicates.
- Dictionary: values accessed by meaningful keys.

4. Functions, modules, and files

Functions

```
def greet(name):
    return f"Hello, {name}!"

message = greet("Sam")
print(message)
```

def defines a function. Parameters receive inputs; return sends a result to the caller. Without an explicit return, a function returns None.

Default arguments

```
def power(number, exponent=2):
    return number ** exponent
print(power(4))     # 16
print(power(2, 3))  # 8
```

Import modules

```
import math
print(math.sqrt(81))
import random
print(random.randint(1, 6))
```

Modules provide reusable code. Python includes a standard library; install third-party packages only when needed.

Read and write a text file

```
with open("notes.txt", "w", encoding="utf-8") as file:
    file.write("Study Python!")
with open("notes.txt", "r", encoding="utf-8") as file:
    text = file.read()
print(text)
```

Modes: r = read, w = write/overwrite, a = append. The with statement closes the file automatically.

Handle expected errors

```
try:
    age = int(input("Age: "))
except ValueError:
    print("Please enter a whole number.")
```

Catch errors you expect and can handle. Avoid hiding every error with a broad, empty except.

5. Debugging and revision

Common errors

Error	Typical cause	Check
SyntaxError	Invalid Python grammar	Quotes, brackets, colons
NameError	Name not defined	Spelling and scope
TypeError	Wrong type for operation	Compatible values or conversion
ValueError	Invalid value	Validate input
IndexError	List index out of range	Valid index / list length
KeyError	Dictionary key missing	Check key or use .get()

Write clearer code

- Use descriptive names, e.g. total_price instead of tp.
- Keep functions focused; avoid repeating the same code.
- Test normal inputs, edge cases, and invalid inputs.
- Read the traceback: it points to the error and often identifies its type.

Practice challenge

Ask the user for three numbers, store them in a list, then print the largest number and the average. Add input validation.

Quick self-test

1. Difference between = and ==?
2. What values does range(2, 6) produce?
3. Why use a function?
4. Which collection stores key–value pairs?
5. What error can converting 'hello' to an integer raise?

Answers

1. Assignment vs comparison.
2. 2, 3, 4, 5.
3. Reuse and organize logic.
4. Dictionary.
5. ValueError.

Next step: build a calculator, quiz, to-do list, or simple expense tracker.